

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №2
з дисципліни «Проектування програмних систем для мобільних пристроїв»

«Шаблони проектування для мобільних пристроїв»

Варіант №4

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 08.10.2026

Мета. Вивчити основні шаблони проектування і розглянути відносини та взаємодії між класами або об'єктами в них.

Завдання. Розробити Android додаток з використанням архітектури MVC, що виконує введення, виведення та редагування даних відповідно до завдання за варіантом. Для виконання кожного пункту завдання (а-в) використовувати окрему Activity та модель. Запустити програму на емуляторі.

Завдання за варіантом. Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім'я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об'єктів.

Вивести:

- а. список абітурієнтів, які мають незадовільні оцінки;
- б. список абітурієнтів, середній бал у яких вище заданого;
- в. вибрати задане число N абітурієнтів, що мають найбільш високій середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

Хід роботи.

Для виконання роботи було повторно використано базовий стек з першого завдання – Rust, Tauri, React з TypeScript. Проєкт Lab2 створено копіюванням структури Lab1 з подальшою заміною всіх ідентифікаторів: назви пакету в Cargo.toml та package.json, productName та identifier (com.xairaven.lab2) у tauri.conf.json, а також простору імен Android-пакету.

Головна складність завдання полягала в поєднанні двох вимог: архітектури MVC та використання окремої Activity і моделі для кожного пункту завдання. Проблема в тому, що Activity – це конкретний клас платформи Android, а застосунок на Tauri технічно складається з однієї-єдиної Activity, яка монтує системний WebView. Кількох справжніх Activity в межах одного Tauri-застосунку одночасно не існує. Було прийнято рішення не імітувати це лише назвою (називаючи React-компоненти «Activity», хоча вони нею не є), а відтворити ті механізми, які Activity реально виконує: окремий, адресований власним URL екран із власним стеком навігації та власну пару Controller+Model, яка не ділиться даними з іншими екранами напрямку.

Реалізована архітектура:

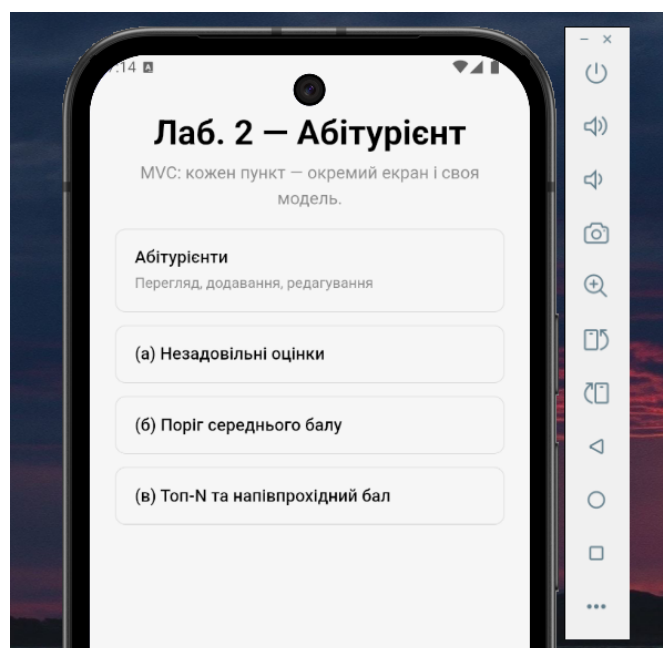
- **Model** – applicant.rs (сутність Applicant та чисті функції бізнес-логіки: average_grade, has_failing_grade, is_above_average, is_borderline) та

новий, порівняно з першою роботою, файл `repository.rs` – спільне мутабельне сховище під `Mutex`, зареєстроване як керований стан `Tauri` (`tauri::State`). Якщо в Лаб.1 було достатньо щоразу генерувати свіжий масив даних (оскільки він лише читався), то Лаб.2 вимагає редагування, тож дані повинні зберігати стан між викликами різних екранів.

- **Controller** – модуль `screen`, що містить чотири підмодулі: `failing`, `threshold`, `top` та `students`. Кожен підмодуль – це одна чи декілька функцій `#[tauri::command]`, яка отримує запит від `View`, звертається до `Model` і повертає дані у форматі, призначеному саме для свого екрану (наприклад, `TopNResult { top, borderline }` – структура, про яку `Model` нічого не знає).
- **View** – компоненти `src/screens/*.tsx`: `Home.tsx` (головний екран-меню, аналог лаунчера застосунку), `Failing.tsx`, `Threshold.tsx`, `TopN.tsx` – по одному на кожен пункт (а-в), та `Students.tsx` – екран введення, виведення й редагування. Компоненти не містять жодної бізнес-логіки, лише відображення та виклик свого `Controller` через `invoke()`.

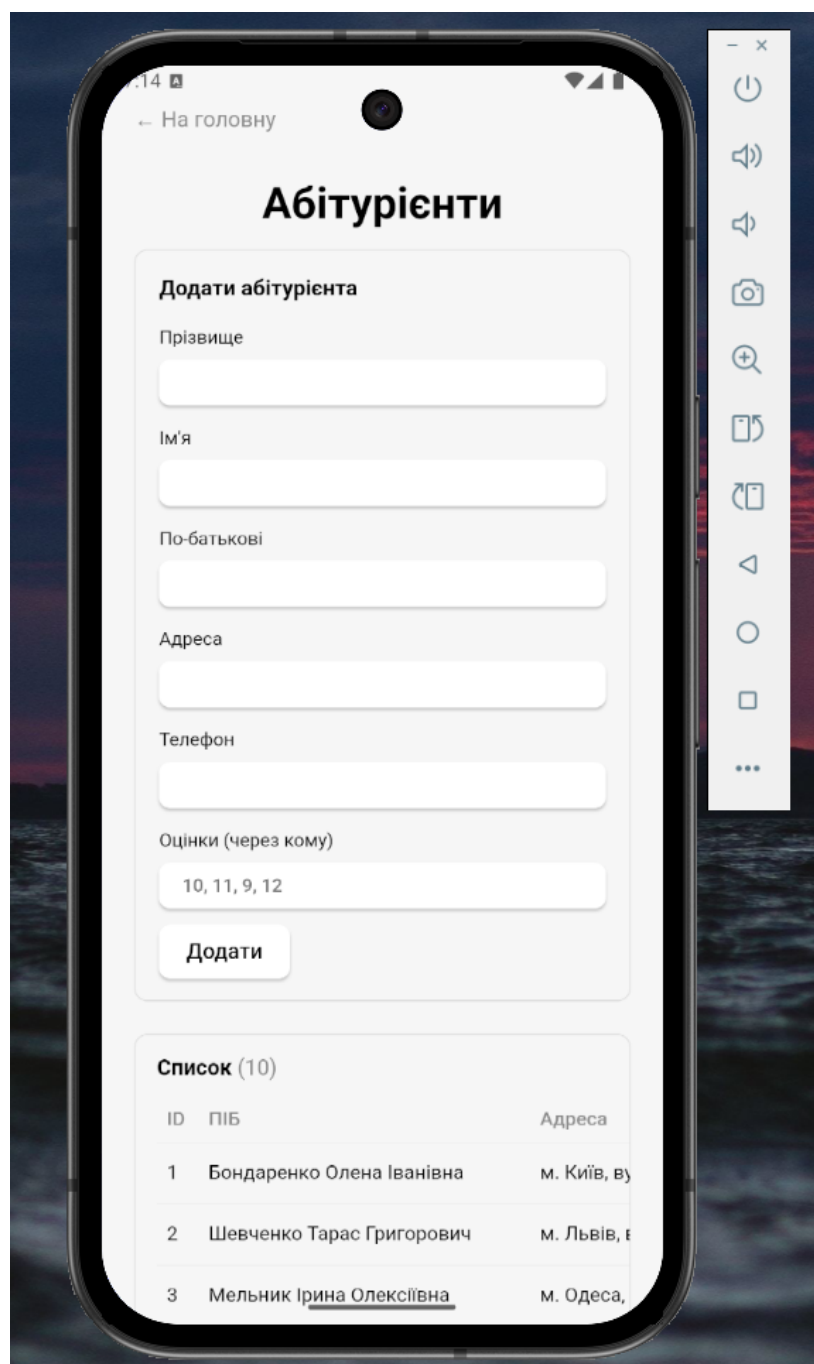
Для навігації між екранами використано бібліотеку `react-router-dom` у режимі `HashRouter`. Це надає кожному екрану справжню URL-адресу та додає його до історії браузера – найближчий доступний у межах одного `WebView` аналог стеку викликів `startActivity()/finish()` в `Android`.

Введення та редагування реалізовано через команди `add_applicant` і `update_applicant`, що приймають DTO `ApplicantInput` (без поля `id` – його призначає сховище). Команда редагування повертає `Result<ApplicantView, String>`: якщо абітурієнта з вказаним ID не існує, помилка передається через IPC та потрапляє у відхилений `Promise` на фронтенді, де відображається користувачу.



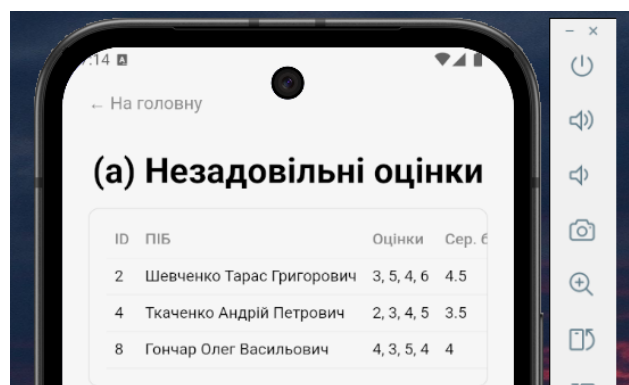
Повторно застосовано суворий набір лінтів Clippy, налаштований у Cargo.toml ще в Лаб.1 (pedantic та nursery на рівні попередження, а unwrap_used, expect_used, panic, indexing_slicing та unsafe_code – заборонено повністю). Це змусило, наприклад, реалізувати блокування Mutex через unwrap_or_else(PoisonError::into_inner) замість unwrap().

Під час ініціалізації Android-проєкту для Лаб.2 довелося знову зіткнутися з помилкою rnpn-native, описаною в звіті з Лаб.1. Цього разу, однак, виявилось, що вона вже виправлена розробниками Tauri в релізі @tauri-apps/cli версії 2.12.0 (мій Pull Request, згаданий у звіті з Лаб.1, був прийнятий саме в цьому релізі) – тож після оновлення залежності ручний патч згенерованого файлу BuildTask.kt більше не знадобився.

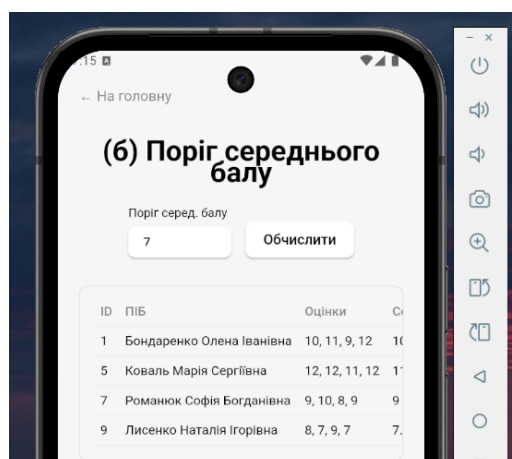


Під час перевірки застосунку на емуляторі виявлено й виправлено дві проблеми.

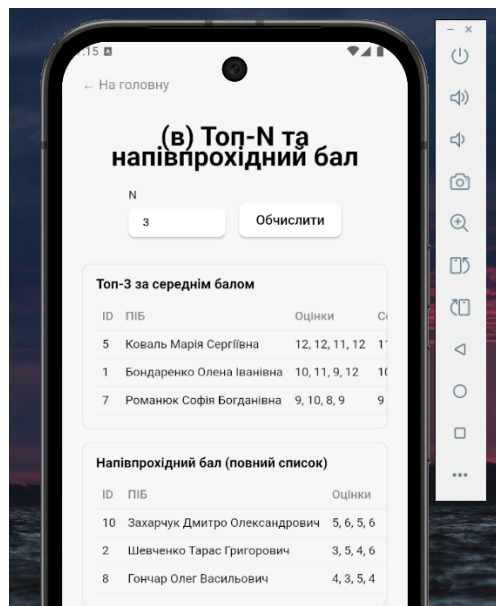
По-перше, таблиця зі списком абітурієнтів не вміщувалась у ширину екрану, але й не прокручувалась горизонтально. Причиною виявилось правило CSS `table { width: 100%; }` – воно змушувало браузер стискати вміст комірок (переносити текст на новий рядок), а не виходити за межі контейнера, тому контейнеру з `overflow-x: auto` просто не було що прокручувати. Виправлення: заміна `width` на `min-width: 100%` та додавання `white-space: nowrap` для комірок – тепер таблиця на вузькому екрані справді виходить за межі контейнера, і цей контейнер стає прокручуваним.



По-друге, кирилицю не вдавалось ввести з фізичної клавіатури хоста. Причина в тому, що емулятор пересилає натискання клавіш як натискання *апаратної* клавіатури, яку Android інтерпретує через власну внутрішню розкладку, а не через системну IME (на кшталт Gboard) – і ця апаратна емуляція переважно розрахована на латиницю. Крім того, коли Android вважає, що підключена апаратна клавіатура, він автоматично приховує програмну клавіатуру. Проблема вирішено вимкненням пересилання апаратної клавіатури в налаштуваннях емулятора (Extended Controls – Settings – General), після чого при фокусі на полі вводу з'являється Gboard, якому достатньо додати українську розкладку.



Отже, принцип MVC у роботі реалізовано так: Model (applicant.rs + repository.rs) не має жодного уявлення про Tauri, IPC чи React – її можна покрити модульними тестами (cargo test) без жодної участі інтерфейсу. Controller (модуль screen) – єдина ланка, яка знає і про Model, і про формат, потрібний конкретному View; саме тут відбувається «переклад» внутрішніх даних у структуру, зручну для відображення. View (компоненти src/screens) не містить обчислень – лише розмітку та виклик Controller. Пункти завдання (а-в) отримали кожен свою пару Controller+Model-представлення (screen::failing, screen::threshold та screen::top із власною структурою TopNResult), що є функціональним еквівалентом вимоги «окрема Activity і модель на кожен пункт» у межах однієї WebView-Activity, властивій Tauri.



Висновок. Під час виконання лабораторної роботи розроблено Android-додаток за доменною областю «Абітурієнт» з використанням архітектурного шаблону MVC. Оскільки обраний стек (Tauri) не передбачає кількох справжніх Activity в межах одного застосунку, вимогу «окрема Activity і модель на кожен пункт завдання» реалізовано функціонально: кожен пункт (а-в) отримав власний маршрут (екран) з власним стеком навігації, власну Controller-команду на боці Rust та власну модель-представлення даних. Реалізовано повний цикл введення, виведення та редагування даних абітурієнтів через окремий екран зі спільним мутабельним станом (Repository), захищеним Mutex. Застосунок успішно скомпільовано та перевірено на емуляторі Android; у процесі перевірки виявлено й виправлено помилку стилів, через яку таблиця не прокручувалась горизонтально на вузьких екранах.

Контрольні запитання.

1. Дайте визначення поняття «патерн»

Патерн (шаблон проєктування) – це узагальнений, перевірений практикою опис типового рішення задачі, яка регулярно виникає в певному контексті розробки програмного забезпечення. Патерн не є готовим фрагментом коду – це опис структури класів чи об'єктів та їхньої взаємодії, який адаптується під конкретну мову програмування та конкретну задачу. Патерни фіксують накопичений досвід спільноти розробників і дозволяють використовувати перевірені рішення замість винаходу власних щоразу заново.

2. Дайте характеристику шаблону Model – View – Controller (MVC)

MVC – архітектурний шаблон, що розділяє застосунок на три взаємопов'язані компоненти. Model відповідає за дані та бізнес-логіку, не маючи жодного уявлення про інтерфейс користувача. View відповідає лише за відображення даних. Controller приймає дії користувача (через View), перетворює їх на звернення до Model і за потреби оновлює View відповідно до нового стану Model. Класична схема MVC допускає, що View може напряду спостерігати за змінами Model (патерн «Спостерігач»), через що межа між View та Controller на практиці часто розмивається – особливо в Android, де роль Controller і частково View традиційно виконує один і той самий клас Activity.

3. Дайте характеристику шаблону Model – View – Presenter (MVP)

MVP – варіація MVC, у якій View повністю пасивний і ніколи не звертається до Model напряду. Весь зв'язок між Model і View проходить через Presenter: View через інтерфейс повідомляє Presenter про дії користувача, Presenter звертається до Model, отримує результат і явно викликає методи View для оновлення відображення. Оскільки View описаний інтерфейсом, а не конкретним класом платформи, Presenter легко покрити модульними тестами без залучення реального UI – це головна практична перевага MVP над MVC, зокрема в класичній розробці під Android.

4. Дайте характеристику шаблону Model – View – ViewModel (MVVM).

MVVM – шаблон, у якому ViewModel взагалі не тримає посилання на View (на відміну від Presenter у MVP). Натомість ViewModel надає спостережуваний стан (наприклад, LiveData/StateFlow в Android Jetpack, чи Observable в інших системах), на який View підписується самостійно через механізм прив'язки даних (data binding). Дії користувача View передає у ViewModel

як виклики методів чи команди. Оскільки ViewModel не знає про існування конкретного View, одна ViewModel потенційно може обслуговувати декілька різних View, а зв'язок стає ще слабшим, ніж у MVP.

5. Перелічіть проблеми, з якими стикається шаблон MVC.

Основна проблема MVC – нечітка межа між View і Controller, яка на практиці часто призводить до того, що один клас виконує обидві ролі (класичний приклад – «роздута Activity» в Android, що бере на себе і обробку дій користувача, і логіку відображення). Це ускладнює модульне тестування, оскільки Controller виявляється прив'язаним до класів платформи, які неможливо інстанціювати поза реальним UI-середовищем. Крім того, за прямого спостереження View за Model зв'язки стають двонаправленими й заплутаними зі зростанням застосунку, а Controller зі збільшенням кількості екранів схильний накопичувати дедалі більше непов'язаної між собою логіки.

6. Назвіть основні принципи шаблону HMVC.

HMVC (Hierarchical Model – View – Controller) будується на кількох принципах: ієрархічна організація застосунку у вигляді дерева незалежних MVC-трійок замість однієї монолітної трійки на весь застосунок; інкапсуляція кожної трійки – вона не знає про внутрішню реалізацію інших, а взаємодіє з ними лише через їхні контролери, як із «чорною скринькою»; контролер як єдина точка входу для своєї трійки, через яку проходять усі запити ззовні; можливість повторно використовувати цілу трійку Model–View–Controller як самостійний, незалежний модуль в інших частинах застосунку чи навіть в інших проєктах.

7. Назвіть види HMVC.

HMVC здебільшого поділяють за рівнем, на якому формується ієрархія трійок. На рівні модулів (module-level HMVC) кожна незалежна функціональна частина застосунку (наприклад, окремий розділ чи фіча) є власною MVC-трійкою, яку головний (front) контролер лише викликає та комбінує результати. На рівні віджетів (widget-level HMVC) окремою трійкою стає кожен UI-компонент, що дозволяє комбінувати незалежні, повторно використовувані фрагменти інтерфейсу в межах однієї сторінки.

Лістинг.

main.rs

```
1 // Prevents additional console window on Windows in release, DO NOT REMOVE!!
2 #![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
3
4 fn main() {
5     lab2_lib::run();
6 }
```

lib.rs

```
1 mod applicant;
2 mod repository;
3 mod sample;
4 mod screen;
5
6 use repository::Repository;
7 use std::sync::Mutex;
8
9 #[cfg_attr(mobile, tauri::mobile_entry_point)]
10 pub fn run() {
11     if let Err(err) = tauri::Builder::default()
12         .plugin(tauri_plugin_opener::init())
13         .manage(Mutex::new(Repository::new()))
14         .invoke_handler(tauri::generate_handler![
15             screen::failing::get_failing,
16             screen::threshold::get_above_threshold,
17             screen::top::get_top_n,
18             screen::students::list_applicants,
19             screen::students::add_applicant,
20             screen::students::update_applicant,
21         ])
22         .run(tauri::generate_context!())
23     {
24         eprintln!("Error occurred while running Tauri application: {err}");
25         std::process::exit(1);
26     }
27 }
```

applicant.rs

```
1 use serde::{Deserialize, Serialize};
2
3 #[derive(Debug, Clone, Serialize)]
4 pub struct Applicant {
5     pub id: u32,
6     pub last_name: String,
7     pub first_name: String,
8     pub patronymic: String,
9     pub address: String,
10    pub phone: String,
11    pub grades: Vec<u8>,
12 }
13
14 pub const PASSING_GRADE: u8 = 4;
15 pub const BORDERLINE_RANGE: std::ops::Range<f64> = 4.0..6.0;
16
17 impl Applicant {
18     pub fn average_grade(&self) -> f64 {
```

```

19         if self.grades.is_empty() {
20             return 0.0;
21         }
22         let sum = f64::from(self.grades.iter().sum::<u8>());
23         #[allow(clippy::cast_precision_loss)]
24         let avg = sum / self.grades.len() as f64;
25
26         (avg * 100.0).round() / 100.0
27     }
28
29     pub fn has_failing_grade(&self) -> bool {
30         self.grades.iter().any(|&g| g < PASSING_GRADE)
31     }
32
33     pub fn is_above_average(&self, threshold: f64) -> bool {
34         self.average_grade() > threshold
35     }
36
37     pub fn is_borderline(&self) -> bool {
38         BORDERLINE_RANGE.contains(&self.average_grade())
39     }
40 }
41
42 #[derive(Debug, Clone, Serialize)]
43 pub struct ApplicantView {
44     #[serde(flatten)]
45     pub applicant: Applicant,
46     pub average: f64,
47 }
48
49 impl From<&Applicant> for ApplicantView {
50     fn from(a: &Applicant) -> Self {
51         Self {
52             applicant: a.clone(),
53             average: a.average_grade(),
54         }
55     }
56 }
57
58 /// Form payload for adding/editing an applicant (no `id` -- the repository
59 /// assigns that on add, and keeps it unchanged on update).
60 #[derive(Debug, Clone, Deserialize)]
61 pub struct ApplicantInput {
62     pub last_name: String,
63     pub first_name: String,
64     pub patronymic: String,
65     pub address: String,
66     pub phone: String,
67     pub grades: Vec<u8>,
68 }
69
70 impl Applicant {
71     pub fn update_by_input(&mut self, input: ApplicantInput) {
72         self.last_name = input.last_name;
73         self.first_name = input.first_name;
74         self.patronymic = input.patronymic;
75         self.address = input.address;
76         self.phone = input.phone;
77         self.grades = input.grades;
78     }
79 }
80
81 #[cfg(test)]
82 mod tests {

```

```

83 use super::Applicant;
84 use super::BORDERLINE_RANGE;
85 use crate::sample;
86
87 fn applicant(grades: Vec<u8>) -> Applicant {
88     Applicant {
89         id: 0,
90         last_name: "Kovalov".into(),
91         first_name: "Alex".into(),
92         patronymic: "Oleksiyovuch".into(),
93         address: "Somewhere".into(),
94         phone: "Idk?".into(),
95         grades,
96     }
97 }
98
99 #[test]
100 fn average_grade_rounds_to_two_decimals() {
101     assert_eq!(applicant(vec![7, 8, 9]).average_grade(), 8.0);
102     assert_eq!(applicant(vec![5, 6]).average_grade(), 5.5);
103 }
104
105 #[test]
106 fn detects_failing_grades() {
107     assert!(applicant(vec![3, 10, 10]).has_failing_grade());
108     assert!(!applicant(vec![4, 10, 10]).has_failing_grade());
109 }
110
111 #[test]
112 fn filters_above_threshold() {
113     let applicants = sample::get_applicants();
114     let threshold = 9.0;
115
116     let result: Vec<Applicant> = applicants
117         .into_iter()
118         .filter(|a| a.is_above_average(threshold))
119         .collect();
120
121     assert!(result.iter().all(|a| a.average_grade() > 9.0));
122 }
123
124 #[test]
125 fn top_n_is_sorted_descending_and_limited() {
126     let mut applicants = sample::get_applicants();
127     let top_n = 3;
128
129     applicants.sort_by(|a, b| {
130         let a = a.average_grade();
131         let b = b.average_grade();
132         b.total_cmp(&a)
133     });
134     let result: Vec<Applicant> = applicants.into_iter().take(top_n).collect();
135
136     assert_eq!(result.len(), top_n);
137     assert!(result[0].average_grade() >= result[1].average_grade());
138     assert!(result[1].average_grade() >= result[2].average_grade());
139 }
140
141 #[test]
142 fn borderline_stays_within_range() {
143     let applicants = sample::get_applicants();
144
145     let result: Vec<Applicant> = applicants
146         .into_iter()

```

```

147         .filter(|a| a.is_borderline())
148         .collect();
149
150     assert!(
151         result
152             .iter()
153             .all(|a| BORDERLINE_RANGE.contains(&a.average_grade()))
154     );
155 }
156 }

```

repository.rs

```

1  ///! Shared mutable store behind `tauri::State<Mutex<Repository>>`.
2  ///!
3  ///! Lab 1 could rebuild a fresh sample array on every query since it was
4  ///! read-only. Lab 2 needs edits to persist across screens, so this plays
5  ///! the role a singleton repository (or a real database) would play behind
6  ///! several Activities in a native Android MVC app.
7
8  use crate::applicant::{Applicant, ApplicantInput};
9  use crate::sample;
10
11  pub struct Repository {
12      applicants: Vec<Applicant>,
13      next_id: u32,
14  }
15
16  impl Repository {
17      pub fn new() -> Self {
18          let applicants = sample::get_applicants();
19          let next_id = applicants.iter().map(|a| a.id).max().unwrap_or(0) + 1;
20          Self {
21              applicants,
22              next_id,
23          }
24      }
25
26      pub fn list(&self) -> &[Applicant] {
27          &self.applicants
28      }
29
30      pub fn add(&mut self, input: ApplicantInput) -> Applicant {
31          let applicant = Applicant {
32              id: self.next_id,
33              last_name: input.last_name,
34              first_name: input.first_name,
35              patronymic: input.patronymic,
36              address: input.address,
37              phone: input.phone,
38              grades: input.grades,
39          };
40          self.next_id += 1;
41          self.applicants.push(applicant.clone());
42          applicant
43      }
44
45      pub fn update(
46          &mut self, id: u32, input: ApplicantInput,
47      ) -> Result<Applicant, String> {
48          let applicant = self
49              .applicants
50              .iter_mut()

```

```

51         .find(|a| a.id == id)
52         .ok_or_else(|| format!("Абітурієнта з ID {id} не знайдено"))?;
53
54     applicant.update_by_input(input);
55
56     Ok(applicant.clone())
57 }
58 }
59
60 impl Default for Repository {
61     fn default() -> Self {
62         Self::new()
63     }
64 }

```

sample.rs

```

1 use crate::applicant::Applicant;
2
3 pub fn get_applicants() -> Vec<Applicant> {
4     vec![
5         Applicant {
6             id: 1,
7             last_name: "Бондаренко".into(),
8             first_name: "Олена".into(),
9             patronymic: "Іванівна".into(),
10            address: "м. Київ, вул. Хрещатик, 10".into(),
11            phone: "+380501112233".into(),
12            grades: vec![10, 11, 9, 12],
13        },
14        Applicant {
15            id: 2,
16            last_name: "Шевченко".into(),
17            first_name: "Тарас".into(),
18            patronymic: "Григорович".into(),
19            address: "м. Львів, вул. Франка, 5".into(),
20            phone: "+380671234567".into(),
21            grades: vec![3, 5, 4, 6],
22        },
23        Applicant {
24            id: 3,
25            last_name: "Мельник".into(),
26            first_name: "Ірина".into(),
27            patronymic: "Олексіївна".into(),
28            address: "м. Одеса, вул. Дерибасівська, 22".into(),
29            phone: "+380931112244".into(),
30            grades: vec![6, 7, 5, 8],
31        },
32        Applicant {
33            id: 4,
34            last_name: "Ткаченко".into(),
35            first_name: "Андрій".into(),
36            patronymic: "Петрович".into(),
37            address: "м. Харків, вул. Сумська, 15".into(),
38            phone: "+380661234455".into(),
39            grades: vec![2, 3, 4, 5],
40        },
41        Applicant {
42            id: 5,
43            last_name: "Коваль".into(),
44            first_name: "Марія".into(),
45            patronymic: "Сергіївна".into(),
46            address: "м. Дніпро, пр. Яворницького, 30".into(),

```

```

47         phone: "+380501239988".into(),
48         grades: vec![12, 12, 11, 12],
49     },
50     Applicant {
51         id: 6,
52         last_name: "Кравець".into(),
53         first_name: "Віктор".into(),
54         patronymic: "Миколайович".into(),
55         address: "м. Запоріжжя, вул. Соборна, 8".into(),
56         phone: "+380671239900".into(),
57         grades: vec![7, 6, 7, 6],
58     },
59     Applicant {
60         id: 7,
61         last_name: "Романюк".into(),
62         first_name: "Софія".into(),
63         patronymic: "Богданівна".into(),
64         address: "м. Івано-Франківськ, вул. Незалежності, 3".into(),
65         phone: "+380931238877".into(),
66         grades: vec![9, 10, 8, 9],
67     },
68     Applicant {
69         id: 8,
70         last_name: "Гончар".into(),
71         first_name: "Олег".into(),
72         patronymic: "Васильович".into(),
73         address: "м. Полтава, вул. Соборності, 12".into(),
74         phone: "+380661237766".into(),
75         grades: vec![4, 3, 5, 4],
76     },
77     Applicant {
78         id: 9,
79         last_name: "Лисенко".into(),
80         first_name: "Наталія".into(),
81         patronymic: "Ігорівна".into(),
82         address: "м. Чернігів, вул. Шевченка, 7".into(),
83         phone: "+380501236655".into(),
84         grades: vec![8, 7, 9, 7],
85     },
86     Applicant {
87         id: 10,
88         last_name: "Захарчук".into(),
89         first_name: "Дмитро".into(),
90         patronymic: "Олександрович".into(),
91         address: "м. Вінниця, вул. Соборна, 20".into(),
92         phone: "+380671235544".into(),
93         grades: vec![5, 6, 5, 6],
94     },
95 ]
96 }

```

screen.rs

```

1  pub mod failing;
2  pub mod students;
3  pub mod threshold;
4  pub mod top;

```

screen/failing.rs

```

1  //! Завдання (а): абітурієнти, які мають незадовільні оцінки.

```

```

2
3 // State<T> must be taken by value in every command here -- that's Tauri's
4 // calling convention, not a real ownership need.
5 #[allow(clippy::needless_pass_by_value)]
6
7 use crate::applicant::ApplicantView;
8 use crate::repository::Repository;
9 use std::sync::{Mutex, PoisonError};
10 use tauri::State;
11
12 #[tauri::command]
13 pub fn get_failing(state: State<'_, Mutex<Repository>>) -> Vec<ApplicantView> {
14     let repo = state.lock().unwrap_or_else(PoisonError::into_inner);
15     repo.list()
16         .iter()
17         .filter(|a| a.has_failing_grade())
18         .map(ApplicantView::from)
19         .collect()
20 }

```

screen/threshold.rs

```

1 //! Завдання (б): абітурієнти, середній бал у яких вище заданого порогу.
2
3 // State<T> must be taken by value in every command here -- that's Tauri's
4 // calling convention, not a real ownership need.
5 #[allow(clippy::needless_pass_by_value)]
6
7 use crate::applicant::ApplicantView;
8 use crate::repository::Repository;
9 use std::sync::{Mutex, PoisonError};
10 use tauri::State;
11
12 #[tauri::command]
13 pub fn get_above_threshold(
14     state: State<'_, Mutex<Repository>>, threshold: f64,
15 ) -> Vec<ApplicantView> {
16     let repo = state.lock().unwrap_or_else(PoisonError::into_inner);
17     repo.list()
18         .iter()
19         .filter(|a| a.is_above_average(threshold))
20         .map(ApplicantView::from)
21         .collect()
22 }

```

screen/top.rs

```

1 //! Завдання (в): N абітурієнтів з найвищим середнім балом,
2 //! а також повний список тих, хто має напівпрохідний бал.
3
4 // State<T> must be taken by value in every command here -- that's Tauri's
5 // calling convention, not a real ownership need.
6 #[allow(clippy::needless_pass_by_value)]
7
8 use crate::applicant::ApplicantView;
9 use crate::repository::Repository;
10 use serde::Serialize;
11 use std::sync::{Mutex, PoisonError};
12 use tauri::State;
13
14 #[derive(Debug, Clone, Serialize)]

```

```

15 pub struct TopNResult {
16     pub top: Vec<ApplicantView>,
17     pub borderline: Vec<ApplicantView>,
18 }
19
20 #[tauri::command]
21 pub fn get_top_n(state: State<'_, Mutex<Repository>>, n: usize) -> TopNResult {
22     // Clone out of the guard immediately so the lock isn't held while we
23     // sort/filter below.
24     let mut applicants = state
25         .lock()
26         .unwrap_or_else(PoisonError::into_inner)
27         .list()
28         .to_vec();
29     applicants.sort_by(|a, b| b.average_grade().total_cmp(&a.average_grade()));
30
31     let top = applicants.iter().take(n).map(ApplicantView::from).collect();
32     let borderline = applicants
33         .iter()
34         .filter(|a| a.is_borderline())
35         .map(ApplicantView::from)
36         .collect();
37
38     TopNResult { top, borderline }
39 }

```

screen/students.rs

```

1  ///! Введення, виведення та редагування даних: список абітурієнтів + форма
2  ///! додавання/редагування.
3
4  // State<T> must be taken by value in every command here -- that's Tauri's
5  // calling convention, not a real ownership need.
6  #![allow(clippy::needless_pass_by_value)]
7
8  use crate::applicant::{ApplicantInput, ApplicantView};
9  use crate::repository::Repository;
10 use std::sync::{Mutex, PoisonError};
11 use tauri::State;
12
13 #[tauri::command]
14 pub fn list_applicants(state: State<'_, Mutex<Repository>>) -> Vec<ApplicantView> {
15     let repo = state.lock().unwrap_or_else(PoisonError::into_inner);
16     repo.list().iter().map(ApplicantView::from).collect()
17 }
18
19 #[tauri::command]
20 pub fn add_applicant(
21     state: State<'_, Mutex<Repository>>, input: ApplicantInput,
22 ) -> ApplicantView {
23     let mut repo = state.lock().unwrap_or_else(PoisonError::into_inner);
24     ApplicantView::from(&repo.add(input))
25 }
26
27 #[tauri::command]
28 pub fn update_applicant(
29     state: State<'_, Mutex<Repository>>, id: u32, input: ApplicantInput,
30 ) -> Result<ApplicantView, String> {
31     let mut repo = state.lock().unwrap_or_else(PoisonError::into_inner);
32     repo.update(id, input).map(|a| ApplicantView::from(&a))
33 }

```

types.ts

```

1 // Mirrors src-tauri/src/applicant.rs and the per-screen result shapes --
2 // kept in sync by hand.
3
4 export interface Applicant {
5     id: number;
6     last_name: string;
7     first_name: string;
8     patronymic: string;
9     address: string;
10    phone: string;
11    grades: number[];
12    average: number;
13 }
14
15 // Form payload for add/edit -- no id/average, those are server-assigned.
16 export interface ApplicantInput {
17     last_name: string;
18     first_name: string;
19     patronymic: string;
20     address: string;
21     phone: string;
22     grades: number[];
23 }
24
25 export interface TopNResult {
26     top: Applicant[];
27     borderline: Applicant[];
28 }

```

main.tsx

```

1 import React from "react";
2 import ReactDOM from "react-dom/client";
3 import App from "./App";
4
5 ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(
6   <React.StrictMode>
7     <App />
8   </React.StrictMode>,
9 );

```

App.tsx

```

1 import { HashRouter, Route, Routes } from "react-router-dom";
2 import { Home } from "./screens/Home";
3 import { Students } from "./screens/Students";
4 import { Failing } from "./screens/Failing";
5 import { Threshold } from "./screens/Threshold";
6 import { TopN } from "./screens/TopN";
7 import "./App.css";
8
9 // HashRouter gives each screen a real URL and browser-history back-stack
10 // (the closest available parallel to Android's startActivity/finish() --
11 // Tauri's WebView is a single Activity, so there's no such thing as a
12 // literal second Activity here).
13 function App() {
14     return (
15       <HashRouter>
16         <Routes>
17           <Route path="/" element={<Home />} />
18           <Route path="/students" element={<Students />} />

```

```

19     <Route path="/failing" element={<Failing />} />
20     <Route path="/threshold" element={<Threshold />} />
21     <Route path="/top-n" element={<TopN />} />
22   </Routes>
23 </HashRouter>
24   );
25 }
26
27 export default App;

```

components/ApplicantTable.tsx

```

1  import type { Applicant } from "../types";
2
3  export function ApplicantTable({ rows }: { rows: Applicant[] }) {
4    if (rows.length === 0) {
5      return <p className="empty">❌ немає ❌</p>;
6    }
7
8    return (
9      <table>
10        <thead>
11          <tr>
12            <th>ID</th>
13            <th>ПІБ</th>
14            <th>Оцінки</th>
15            <th>Сер. бал</th>
16          </tr>
17        </thead>
18        <tbody>
19          {rows.map((a) => (
20            <tr key={a.id}>
21              <td>{a.id}</td>
22              <td>
23                {a.last_name} {a.first_name} {a.patronymic}
24              </td>
25              <td>{a.grades.join(", ")}</td>
26              <td>{a.average}</td>
27            </tr>
28          ))}
29        </tbody>
30      </table>
31    );
32  }

```

screens/Home.tsx

```

1  import { Link } from "react-router-dom";
2
3  export function Home() {
4    return (
5      <main className="container">
6        <h1>Лаб. 2 ❌ Абітурієнт</h1>
7        <p className="subtitle">MVC: кожен пункт ❌ окремий екран і своя модель.</p>
8
9        <nav className="menu">
10          <Link className="menu-item" to="/students">
11            Абітурієнти
12          <span>Перегляд, додавання, редагування</span>
13          </Link>
14          <Link className="menu-item" to="/failing">

```

```

15     (a) Незадовільні оцінки
16   </Link>
17   <Link className="menu-item" to="/threshold">
18     (б) Попіг середнього балу
19   </Link>
20   <Link className="menu-item" to="/top-n">
21     (в) Топ-N та напівпрохідний бал
22   </Link>
23 </nav>
24 </main>
25   );
26 }

```

screens/Failing.tsx

```

1  import { useEffect, useState } from "react";
2  import { invoke } from "@tauri-apps/api/core";
3  import { Link } from "react-router-dom";
4  import { ApplicantTable } from "../components/ApplicantTable";
5  import type { Applicant } from "../types";
6
7  export function Failing() {
8    const [rows, setRows] = useState<Applicant[] | null>(null);
9
10    useEffect(() => {
11      invoke<Applicant[]>("get_failing").then(setRows);
12    }, []);
13
14    return (
15      <main className="container">
16        <Link className="back" to="/">
17          На головну
18        </Link>
19        <h1>(a) Незадовільні оцінки</h1>
20        <section className="panel">{rows && <ApplicantTable rows={rows} />}</section>
21      </main>
22    );
23  }

```

screens/Threshold.tsx

```

1  import { useEffect, useState } from "react";
2  import { invoke } from "@tauri-apps/api/core";
3  import { Link } from "react-router-dom";
4  import { ApplicantTable } from "../components/ApplicantTable";
5  import type { Applicant } from "../types";
6
7  export function Threshold() {
8    // Kept as a string (not a number) so the input always shows exactly
9    // what was typed -- a controlled number-input round-trip otherwise
10   // produces a "05"-style leading-zero glitch.
11   const [thresholdInput, setThresholdInput] = useState("7");
12   const [rows, setRows] = useState<Applicant[] | null>(null);
13
14   async function compute() {
15     const threshold = Number(thresholdInput) || 0;
16     setRows(await invoke<Applicant[]>("get_above_threshold", { threshold }));
17   }
18
19   useEffect(() => {
20     compute();

```

```

21     }, []);
22
23     return (
24       <main className="container">
25         <Link className="back" to="/">
26           На головну
27         </Link>
28         <h1>(6) Попіг середнього балу</h1>
29
30         <form
31           className="controls"
32           onSubmit={(e) => {
33             e.preventDefault();
34             compute();
35           }}
36         >
37           <label>
38             Попіг серед. балу
39           <input
40             type="number"
41             step="0.1"
42             value={thresholdInput}
43             onChange={(e) => setThresholdInput(e.currentTarget.value)}
44           />
45           </label>
46           <button type="submit">Обчислити</button>
47         </form>
48
49         <section className="panel">{rows && <ApplicantTable rows={rows} />}</section>
50       </main>
51     );
52   }

```

screens/TopN.tsx

```

1   import { useEffect, useState } from "react";
2   import { invoke } from "@tauri-apps/api/core";
3   import { Link } from "react-router-dom";
4   import { ApplicantTable } from "../components/ApplicantTable";
5   import type { TopNResult } from "../types";
6
7   export function TopN() {
8     const [nInput, setNInput] = useState("3");
9     const [result, setResult] = useState<TopNResult | null>(null);
10
11     async function compute() {
12       const n = Number(nInput) || 0;
13       setResult(await invoke<TopNResult>("get_top_n", { n }));
14     }
15
16     useEffect(() => {
17       compute();
18     }, []);
19
20     return (
21       <main className="container">
22         <Link className="back" to="/">
23           На головну
24         </Link>
25         <h1>(в) Топ-N та напівпрохідний бал</h1>
26
27         <form
28           className="controls"

```

```

29     onSubmit={e => {
30         e.preventDefault();
31         compute();
32     }}
33     >
34     <label>
35     N
36     <input
37     type="number"
38     min={1}
39     value={nInput}
40     onChange={e => setNInput(e.currentTarget.value)}
41     />
42     </label>
43     <button type="submit">Обчислити</button>
44     </form>
45
46     {result && (
47         <>
48         <section className="panel">
49         <h2>Топ-{nInput} за середнім балом</h2>
50         <ApplicantTable rows={result.top} />
51         </section>
52         <section className="panel">
53         <h2>Найімовірніший бал (повний список)</h2>
54         <ApplicantTable rows={result.borderline} />
55         </section>
56         </>
57     )}
58     </main>
59     );
60 }

```

screens/Students.tsx

```

1  import { useEffect, useState, type FormEvent } from "react";
2  import { invoke } from "@tauri-apps/api/core";
3  import { Link } from "react-router-dom";
4  import type { Applicant, ApplicantInput } from "../types";
5
6  const emptyForm = {
7      last_name: "",
8      first_name: "",
9      patronymic: "",
10     address: "",
11     phone: "",
12     gradesText: "",
13 };
14
15 function parseGrades(text: string): number[] {
16     return text
17         .split(/[,\s]+/)
18         .map((token) => Number(token))
19         .filter((n) => Number.isFinite(n) && n >= 0);
20 }
21
22 export function Students() {
23     const [rows, setRows] = useState<Applicant[] | null>(null);
24     const [editingId, setEditingId] = useState<number | null>(null);
25     const [form, setForm] = useState(emptyForm);
26     const [error, setError] = useState<string | null>(null);
27
28     useEffect(() => {

```

```

29     invoke<Applicant[]>("list_applicants").then(setRows);
30 }, []);
31
32 function startEdit(a: Applicant) {
33     setEditingId(a.id);
34     setForm({
35         last_name: a.last_name,
36         first_name: a.first_name,
37         patronymic: a.patronymic,
38         address: a.address,
39         phone: a.phone,
40         gradesText: a.grades.join(", "),
41     });
42     setError(null);
43 }
44
45 function cancelEdit() {
46     setEditingId(null);
47     setForm(emptyForm);
48     setError(null);
49 }
50
51 async function submit(e: FormEvent) {
52     e.preventDefault();
53     const input: ApplicantInput = {
54         last_name: form.last_name,
55         first_name: form.first_name,
56         patronymic: form.patronymic,
57         address: form.address,
58         phone: form.phone,
59         grades: parseGrades(form.gradesText),
60     };
61
62     try {
63         if (editingId === null) {
64             const created = await invoke<Applicant>("add_applicant", { input });
65             setRows((prev) => (prev ? [...prev, created] : [created]));
66         } else {
67             const updated = await invoke<Applicant>("update_applicant", {
68                 id: editingId,
69                 input,
70             });
71             setRows((prev) =>
72                 prev ? prev.map((a) => (a.id === editingId ? updated : a)) : prev,
73             );
74         }
75         cancelEdit();
76     } catch (err) {
77         setError(String(err));
78     }
79 }
80
81 return (
82     <main className="container">
83     <Link className="back" to="/">
84     
85     </Link>
86     <h1>Абітурієнти</h1>
87
88     <form className="form" onSubmit={submit}>
89     <h2>{editingId === null ? "Додати абітурієнта" : `Редагувати ID ${editingId}`}</h2>
90     {error && <p className="error">{error}</p>}
91     <label>
92     Прізвище

```

```

93     <input
94     required
95     value={form.last_name}
96     onChange={(e) => setForm({ ...form, last_name: e.currentTarget.value })}
97     />
98   </label>
99   <label>
100   Ім'я
101   <input
102   required
103   value={form.first_name}
104   onChange={(e) => setForm({ ...form, first_name: e.currentTarget.value })}
105   />
106   </label>
107   <label>
108   По-батькові
109   <input
110   value={form.patronymic}
111   onChange={(e) => setForm({ ...form, patronymic: e.currentTarget.value })}
112   />
113   </label>
114   <label>
115   Адреса
116   <input
117   value={form.address}
118   onChange={(e) => setForm({ ...form, address: e.currentTarget.value })}
119   />
120   </label>
121   <label>
122   Телефон
123   <input
124   value={form.phone}
125   onChange={(e) => setForm({ ...form, phone: e.currentTarget.value })}
126   />
127   </label>
128   <label>
129   Оцінки (через кому)
130   <input
131   placeholder="10, 11, 9, 12"
132   value={form.gradesText}
133   onChange={(e) => setForm({ ...form, gradesText: e.currentTarget.value })}
134   />
135   </label>
136   <div className="form-actions">
137   <button type="submit">{editingId === null ? "Додати" : "Зберегти"}</button>
138   {editingId !== null && (
139     <button type="button" onClick={cancelEdit}>
140     Скасувати
141     </button>
142   )}
143   </div>
144   </form>
145
146   <section className="panel">
147   <h2>
148   Список <span className="count">({rows?.length ?? 0})</span>
149   </h2>
150   {rows && rows.length > 0 ? (
151     <table>
152     <thead>
153     <tr>
154     <th>ID</th>
155     <th>ПІБ</th>
156     <th>Адреса</th>

```

```

157     <th>Телефон</th>
158     <th>Оцінки</th>
159     <th>Сер. бал</th>
160     <th></th>
161   </tr>
162 </thead>
163 <tbody>
164   {rows.map((a) => (
165     <tr key={a.id}>
166       <td>{a.id}</td>
167       <td>
168         {a.last_name} {a.first_name} {a.patronymic}
169       </td>
170       <td>{a.address}</td>
171       <td>{a.phone}</td>
172       <td>{a.grades.join(", ")}</td>
173       <td>{a.average}</td>
174       <td>
175         <button type="button" onClick={() => startEdit(a)}>
176           Редагувати
177         </button>
178       </td>
179     </tr>
180   ))}
181 </tbody>
182 </table>
183   ) : (
184     <p className="empty">❏ немає ❏</p>
185   )}
186 </section>
187 </main>
188   );
189 }

```

App.css

```

1  :root {
2    font-family: Inter, Avenir, Helvetica, Arial, sans-serif;
3    font-size: 16px;
4    line-height: 24px;
5    font-weight: 400;
6
7    color: #0f0f0f;
8    background-color: #f6f6f6;
9
10   font-synthesis: none;
11   text-rendering: optimizeLegibility;
12   -webkit-font-smoothing: antialiased;
13   -moz-osx-font-smoothing: grayscale;
14   -webkit-text-size-adjust: 100%;
15 }
16
17 .container {
18   margin: 0 auto;
19   max-width: 720px;
20   padding: 2rem 1rem 4rem;
21 }
22
23 h1 {
24   text-align: center;
25 }
26
27 .subtitle {

```

```

28     text-align: center;
29     color: #8a8a8a;
30     margin-top: -0.5rem;
31 }
32
33 .back {
34     display: inline-block;
35     color: #8a8a8a;
36     text-decoration: none;
37     margin-bottom: 1rem;
38 }
39
40 .back:hover {
41     color: #396cd8;
42 }
43
44 .menu {
45     display: flex;
46     flex-direction: column;
47     gap: 0.75rem;
48 }
49
50 .menu-item {
51     display: flex;
52     flex-direction: column;
53     border: 1px solid #dcdcdc;
54     border-radius: 10px;
55     padding: 0.9rem 1.1rem;
56     color: inherit;
57     text-decoration: none;
58     font-weight: 500;
59 }
60
61 .menu-item:hover {
62     border-color: #396cd8;
63 }
64
65 .menu-item span {
66     font-weight: 400;
67     font-size: 0.85em;
68     color: #8a8a8a;
69 }
70
71 .form {
72     display: flex;
73     flex-direction: column;
74     gap: 0.75rem;
75     border: 1px solid #dcdcdc;
76     border-radius: 10px;
77     padding: 1rem 1.1rem;
78     margin-bottom: 1.5rem;
79 }
80
81 .form h2 {
82     font-size: 1em;
83     margin: 0;
84 }
85
86 .form label {
87     display: flex;
88     flex-direction: column;
89     gap: 0.25rem;
90     font-size: 0.85em;
91 }

```

```

92
93 .form input {
94     width: 100%;
95     box-sizing: border-box;
96 }
97
98 .form-actions {
99     display: flex;
100     gap: 0.5rem;
101 }
102
103 .error {
104     color: #d1453b;
105     margin: 0;
106 }
107
108 .controls {
109     display: flex;
110     flex-wrap: wrap;
111     align-items: end;
112     gap: 1rem;
113     justify-content: center;
114     margin-bottom: 2rem;
115 }
116
117 .controls label {
118     display: flex;
119     flex-direction: column;
120     gap: 0.25rem;
121     font-size: 0.85em;
122 }
123
124 input,
125 button {
126     border-radius: 8px;
127     border: 1px solid transparent;
128     padding: 0.6em 1.2em;
129     font-size: 1em;
130     font-weight: 500;
131     font-family: inherit;
132     color: #0f0f0f;
133     background-color: #ffffff;
134     transition: border-color 0.25s;
135     box-shadow: 0 2px 2px rgba(0, 0, 0, 0.2);
136     outline: none;
137 }
138
139 input {
140     width: 6em;
141 }
142
143 button {
144     cursor: pointer;
145 }
146
147 button:hover {
148     border-color: #396cd8;
149 }
150 button:active {
151     border-color: #396cd8;
152     background-color: #e8e8e8;
153 }
154
155 .panel {

```

```

156     border: 1px solid #dcdcdc;
157     border-radius: 10px;
158     padding: 0.75rem 1rem 1rem;
159     margin-bottom: 1rem;
160     overflow-x: auto;
161     -webkit-overflow-scrolling: touch;
162 }
163
164 .panel h2 {
165     font-size: 1em;
166     margin: 0 0 0.6rem;
167 }
168
169 .count {
170     color: #8a8a8a;
171     font-weight: 400;
172 }
173
174 .empty {
175     color: #8a8a8a;
176     margin: 0;
177 }
178
179 table {
180     /* min- not plain width: lets the table grow past the panel on narrow
181        screens instead of cramming/wrapping cells to fit -- that's what makes
182        .panel's overflow-x: auto actually have something to scroll. */
183     min-width: 100%;
184     border-collapse: collapse;
185     font-size: 0.9em;
186 }
187
188 th,
189 td {
190     text-align: left;
191     padding: 0.3em 0.5em;
192     border-bottom: 1px solid #eaeaea;
193     white-space: nowrap;
194 }
195
196 th {
197     color: #8a8a8a;
198     font-weight: 500;
199 }
200
201 @media (prefers-color-scheme: dark) {
202     :root {
203         color: #f6f6f6;
204         background-color: #2f2f2f;
205     }
206
207     input,
208     button {
209         color: #ffffff;
210         background-color: #0f0f0f98;
211     }
212     button:active {
213         background-color: #0f0f0f69;
214     }
215
216     .panel,
217     .menu-item,
218     .form {
219         border-color: #4a4a4a;

```

```
220     }
221
222     th,
223     td {
224         border-bottom-color: #3f3f3f;
225     }
226 }
```